MCTS Project

Gabriella FERNANDES MACEDO
Ruben IFRAH

Contents

1 Reward Function Design and Optimization Strategy **2**
1.1 The Baseline Formulation and its Limitations 2
1.2 Transition to a Hybrid MCTS-SINDy Architecture 2
1.3 The Intermediate Additive Penalty Reward 2
1.4 Final Formulation: Information Theory 3
1.5 Structural Complexity and the Hyperparameter Dichotomy 3

2 Grammar and Tree Construction **4**
2.1 Production Rules 4
2.2 LIFO Stack Mechanics 5
2.3 AST Construction Example 5

3 Advanced Exploration Strategies **6**
3.1 Current Approach: Vanilla UCT with Uniform Random Rollouts 6
3.2 SHUSS (Sequential Halving Using Scores for Search) 7
3.3 AMAF (All-Moves-As-First) & RAVE 7
3.4 RL-Guided Policy Networks 7

1 Motivation: Why Combine MCTS and SINDy?

The fundamental goal of Symbolic Regression (SR) is to automatically discover mathematical equations that govern a dataset. Traditionally, this is achieved either through evolutionary algorithms (like Genetic Programming) or through sparse regression over a predefined dictionary of functions (like SINDy). However, both approaches suffer from fatal scaling barriers when applied to highly complex dynamical systems. The proposed hybrid MCTS-SINDy architecture is specifically designed to bypass these two distinct barriers.

1.1 The Continuous Optimization Bottleneck of Standard SR

In traditional grammar-guided Symbolic Regression (such as the original Symbolic Physics Learner), the agent builds a complete algebraic equation tree that includes numerical constants (e.g., $c_0 \sin(x) + c_1 y$). Evaluating the fitness of this tree requires running a continuous, non-convex numerical optimizer (like BFGS or Powell) at the leaf nodes to fit these constants to the data.

This optimization step is computationally devastating. The loss landscapes of arbitrary non-linear equations are riddled with local minima, causing optimizers to fail or take thousands of iterations to converge. This makes standard SR insurmountably slow when scaling to complex multivariate problems.

1.2 The Combinatorial Explosion of Pure SINDy

The SINDy algorithm elegantly solves the constant-fitting problem by framing it as a sparse linear regression task ($\dot{X} = \mathbf{\Theta}(X)\mathbf{\Xi}$). Because it uses Sequentially Thresholded Least Squares (STLSQ) or LASSO, calculating the coefficients $\mathbf{\Xi}$ is practically instantaneous and mathematically convex.

However, pure SINDy relies on a *static, predefined* library of candidate features $\mathbf{\Theta}(X)$ (typically low-degree polynomials and trigonometrics). This works perfectly for simple, well-known systems like the Lorenz attractor. But consider a physical system governed by a deeply nested, highly non-linear term, such as:

$$\dot{x} = \frac{\sin(x^2 \cdot y)}{1 + \exp(-z)} \tag{1}$$

To guarantee that a static dictionary simply *contains* this specific basis function, the researcher would have to forcefully pre-compute every possible mathematical combination of operators $(\sin, \cos, \exp, +, -, *, /)$ and variables (x, y, z) up to depth 6 or 7. Due to the exponential branching factor of mathematical grammars, a depth-7 library consists of millions or billions of generated terms.

Passing a matrix with $p = 1,000,000$ columns and $n = 1,000$ data points to a regression solver creates a massively ill-posed problem ($p \gg n$). The matrices become perfectly singular, STLSQ fails to converge, and the computer runs out of RAM. In short, pure SINDy cannot discover novel governing equations unless the human operator already intuitively knows what non-linear shapes to manually code into the dictionary.

1.3 The Hybrid Solution: Dynamic Memory-Efficient Filtering

The integration of Monte Carlo Tree Search provides the definitive solution to the dictionary explosion problem. MCTS acts as a **dynamic, memory-efficient filter** across the infinite grammar space.

Instead of instantiating 1 billion columns in RAM, the MCTS heuristic traverses the infinite mathematical tree and selectively passes only small, focused feature matrices (e.g., 5 to 10 dynamically generated columns at a time) to SINDy. By leveraging rigorous selection metrics like

the Bayesian Information Criterion (BIC), the MCTS is heavily penalized for exploring garbage branches. This feedback loop allows the agent to autonomously "snipe" the exact non-linear features required to explain the data without ever triggering a dense matrix collapse.

In summary, while using an MCTS to discover simple linear relationships (like $\dot{x} = -0.1x + y$) may feel like overkill compared to static dictionaries, the hybrid architecture is strictly necessary to unlock the autonomous AI discovery of highly nested, counter-intuitive governing equations in fields like fluid dynamics or quantum mechanics that humans have not yet conceptualized.

2 Reward Function Design and Optimization Strategy

The design of the reward function is the most critical component of the Monte Carlo Tree Search (MCTS) architecture, as it provides the sole feedback signal guiding the agent's exploration of the combinatorial grammar space. Our objective is to discover governing physical equations that are both highly accurate and parsimonious. In this section, we detail the evolution of the reward formulation, transitioning from a standard additive penalty model to a robust, information-theoretic framework integrated with sparse regression.

2.1 The Baseline Formulation and its Limitations

Standard grammar-guided symbolic regression frameworks, such as the Symbolic Physics Learner (SPL), evaluate generated expression trees using numerical optimizers (e.g., the Powell method or BFGS) at the leaf nodes to estimate continuous constants. The fitness of a complete tree is typically evaluated using a baseline reward function that balances the Mean Squared Error (MSE) with a structural parsimony penalty:

$$\mathcal{R}_{\text{baseline}} = \frac{\eta^n}{1 + \text{MSE}} \tag{2}$$

where $\eta \in (0, 1)$ is a discount factor and n represents the total number of production rules utilized to construct the tree. While intuitive, this approach suffers from severe computational bottlenecks due to the non-convex nature of the continuous parameter optimization, and it struggles to scale to complex dynamical systems.

2.2 Transition to a Hybrid MCTS-SINDy Architecture

To overcome the computational limitations of non-convex optimization, we propose a hybrid architecture where MCTS acts as a *feature generator* rather than a strict equation builder. The MCTS explores the syntax tree to organically construct a library of candidate nonlinear basis functions, $\mathbf{\Phi}(X) = [\phi_1(X), \phi_2(X), \ldots, \phi_m(X)]$.

This generated library is then passed to a Sequentially Thresholded Least Squares (STLSQ) solver, akin to the SINDy algorithm. The solver instantly computes a sparse vector of coefficients $\mathbf{\Xi}$, forcing the coefficients of redundant or non-governing features to exactly zero. Consequently, the reward function must be redesigned to evaluate not just a single tree, but the performance of the surviving terms within the feature matrix.

2.3 The Intermediate Additive Penalty Reward

In the hybrid paradigm, the reward must penalize complexity, but crucially, it must differentiate between terms that are actively utilized by the STLSQ solver and those that are discarded. An initial proposed formulation for the hybrid reward is an additive model:

$$\mathcal{R}_{\text{additive}} = \frac{1}{1 + \text{MSE} + \lambda\|\mathbf{\Xi}\|_0 + \alpha\mathcal{C}_{\text{active}} + \beta\mathcal{C}_{\text{unused}}} \tag{3}$$

This formulation introduces four competing forces:

- **Data Fidelity (MSE):** The reconstruction error achieved by the STLSQ solver.
- **Sparsity ($\|\boldsymbol{\Xi}\|_0$):** The L_0 norm, directly penalizing the count of non-zero coefficients.
- **Active Complexity ($\mathcal{C}_{\textbf{active}}$):** A heavy structural penalty (α) applied to the grammar depth of the specific features where $\Xi_i \neq 0$.
- **Unused Complexity ($\mathcal{C}_{\textbf{unused}}$):** A minor structural penalty (β) applied to generated features that the solver assigned a coefficient of zero.

A fundamental constraint in this design is that $\beta \ll \alpha$. If β is too large, the MCTS agent becomes risk-averse, proposing only trivial terms to avoid penalties. Conversely, if $\beta = 0$, the agent exploits the solver by proposing infinitely wide feature matrices, leading to ill-conditioned inverse problems. A small β acts as a necessary regularizer, encouraging broad exploration while penalizing gratuitous computational bloat.

2.4 Final Formulation: Information Theory

While the additive reward conceptually bridges tree search and sparse regression, it is mathematically flawed due to scaling discrepancies. The MSE can vary by orders of magnitude across different equations, making the linear sparsity penalty ($\lambda\|\boldsymbol{\Xi}\|_0$) highly unstable and hyperparameter-dependent.

To establish a mathematically rigorous selection criterion, we abandon the linear combination in favor of the Bayesian Information Criterion (BIC). BIC leverages logarithmic scaling to inherently balance the maximum likelihood of the model against its dimensionality:

$$\text{BIC} = n \ln(\text{MSE}) + \|\boldsymbol{\Xi}\|_0 \ln(n) \tag{4}$$

where n is the number of data points. To integrate BIC into the MCTS framework—which requires a bounded reward signal $\mathcal{R} \in (0, 1]$ to maintain calibrated Upper Confidence Bounds (UCT)—we apply an exponential decay to the BIC score, while retaining the structural syntax penalties in the denominator. This yields our final, state-of-the-art reward formulation:

$$\mathcal{R}_{\text{final}} = \exp(-\gamma \cdot \text{BIC}) \times \frac{1}{1 + \alpha\mathcal{C}_{\text{active}} + \beta\mathcal{C}_{\text{unused}}} \tag{5}$$

where γ scales the sensitivity of the exponential. This formulation represents a significant advancement over standard symbolic regression rewards. The numerator statistically guarantees that the MCTS is rewarded for discovering the true governing dynamics without overfitting, while the denominator ensures the mathematical elegance of the underlying syntax tree.

2.5 Structural Complexity and the Hyperparameter Dichotomy

To fully realize the benefits of the hybrid architecture, the denominator of the reward function meticulously decouples the structural complexity of the generated grammar tree into two distinct components: active and unused features.

Let the MCTS agent generate a candidate library comprised of m features, $\boldsymbol{\Phi} = \{\phi_1, \phi_2, \ldots, \phi_m\}$. For each feature ϕ_i, we define its individual structural complexity, $c(\phi_i)$, as the number of discrete production rules required to derive it from the underlying context-free grammar. Upon evaluation by the STLSQ solver, the features are partitioned based on their corresponding sparse coefficients in $\boldsymbol{\Xi}$:

- **Active Complexity ($\mathcal{C}_{\textbf{active}}$):** The sum of grammatical complexities for all features retained by the solver:

$$\mathcal{C}_{\text{active}} = \sum_{i:\Xi_i \neq 0} c(\phi_i) \tag{6}$$

- **Unused Complexity ($\mathcal{C}_{\text{unused}}$):** The sum of grammatical complexities for all features successfully pruned by the solver:

$$\mathcal{C}_{\text{unused}} = \sum_{j:\Xi_j=0} c(\phi_j) \tag{7}$$

A fundamental mathematical constraint for this reward architecture is the strict inequality $\beta \ll \alpha$. This dichotomy resolves the inherent tension between combinatorial exploration and physical parsimony.

If β were assigned a weight comparable to α, the MCTS agent would become highly risk-averse. To avoid severe penalties for generating terms that SINDy might ultimately reject, the optimal policy would collapse into proposing only trivial, low-complexity features (e.g., standard linear variables), effectively neutralizing the MCTS's ability to discover complex, nested dynamics. Conversely, setting $\beta = 0$ removes all friction from generation, allowing the agent to propose infinitely wide feature matrices. This causes the STLSQ optimization to become statistically ill-posed ($p \gg n$) and computationally unstable.

By enforcing $\beta \ll \alpha$, the reward acts as a mild "generation tax." It encourages the MCTS to function as a highly exploratory brainstorming engine—freely proposing highly non-linear, complex basis functions that bypass human bias—while relying on the heavy α penalty to enforce Occam's razor strictly on the final, physically governing terms.

The exponential transformation, $\exp(-\gamma \cdot \text{BIC})$, is a critical adaptation for integrating statistical model selection into a Reinforcement Learning paradigm. Raw Information Criteria can span wide, unbounded numerical ranges depending on the variance of the underlying dataset. Directly backpropagating such unbounded error signals inevitably destabilizes the Upper Confidence Bound (UCT) equation used during the MCTS selection phase.

The hyperparameter γ acts as a temperature scaling factor, compressing the unbounded BIC score into a strictly bounded probability-like domain $(0, 1]$. Consequently, the MCTS agent optimizes over a smooth, normalized manifold. This mathematical synthesis guarantees that the search tree reliably converges toward the Pareto front of models—simultaneously maximizing data likelihood while minimizing both continuous parameter counts and discrete syntactical complexity.

3 Grammar and Tree Construction

The foundation of the MCTS Equation Discovery engine relies on a formal Context-Free Grammar (CFG). Unlike traditional Symbolic Regression that builds a single global algebraic equation (often struggling with optimization of constants), our modified grammar is intrinsically designed to output a *Library of Features*. This decouples the mathematical shape discovery (handled by MCTS) from the linear coefficient optimization (handled by PySINDy).

3.1 Production Rules

The grammar consists of a set of Non-Terminals ($\mathcal{N}$), Terminals ($\mathcal{T}$), and Production Rules ($\mathcal{P}$). The generation always begins with the root symbol f.

- $\mathcal{N} = \{f, \text{Library}, \text{Feature}, M\}$
- $\mathcal{T} = \{x, y, z, *, \sin, \cos, \exp\}$

The core production rules are defined as follows:

$$
\begin{aligned}
f &\to \text{Library} \\
\text{Library} &\to \text{Feature} \\
\text{Library} &\to \text{Feature}, \text{Library} \\
\text{Feature} &\to M \\
M &\to M * M \mid \sin(M) \mid \cos(M) \mid \exp(M) \mid x \mid y \mid z
\end{aligned}
$$

This recursive definition is crucial: the `Library` rule allows the MCTS to dynamically dictate the width of the feature matrix prior to calculating the internal physics of each term. This replaces the rigid "+ / -" operators found in standard equation trees.

3.2 LIFO Stack Mechanics

The MCTS builds the tree sequentially by maintaining a Last-In, First-Out (LIFO) stack of non-terminals. The process begins with the root symbol `f` heavily anchoring the stack. At each step, the agent pops the top non-terminal, selects an applicable production rule, and pushes any newly generated non-terminals onto the stack from right to left (ensuring the leftmost symbol is always expanded next).

Understanding the roles of the specific nodes:

- `f`: The mandatory starting root. It immediately forces the creation of a `Library`.
- `Library`: Dictates the width of the final equation matrix. Calling Library $\to$ Feature, Library adds a new column to the PySINDy matrix, whereas Library $\to$ Feature terminates the matrix expansion.
- `Feature`: An isolation wrapper. It guarantees that the mathematical expression generated underneath it remains a distinct, separable term to be plugged into SINDy, rather than merging algebraically with other terms.
- `M`: The core math generator. It recursively stacks operators $(\sin, \cos, *, \dots)$ and terminates with variables (x, y, z).

3.3 AST Construction Example

Consider the exact sequence of rules chosen to generate Figure 1: a library containing two features $[\phi_1 = x, \phi_2 = \sin(y)]$. The step-by-step mechanics unfold as follows:

1. **Initialize:** Stack $= [f]$
2. **Rule $f \to$ Library:** Pop f, push Library.
   Stack $=$ [Library]
3. **Rule Library $\to$ Feature, Library:** Pop Library, push Library then Feature. The agent decides the feature matrix will have multiple columns.
   Stack $=$ [Feature, Library]
4. **Rule Feature $\to M$:** Pop Feature, push M. We begin building the first basis function.
   Stack $= [M, \text{Library}]$
5. **Rule $M \to x$:** Pop M. As x is a terminal, nothing is pushed.
   Stack $=$ [Library]

6. **Rule Library → Feature:** Pop Library, push Feature. The agent decides this is the final term in the matrix.
   Stack = [Feature]

7. **Rule Feature → M:** Pop Feature, push M.
   Stack = $[M]$

8. **Rule $M \to \sin(M)$:** Pop M, push M. A unary operator is applied.
   Stack = $[M]$

9. **Rule $M \to y$:** Pop M. A terminal is applied.
   Stack = [] (Empty)

When the stack is empty, the sequence is perfectly complete. The parser then extracts all branches originating from a `Feature` node, resulting in the final abstract syntax tree shown below.

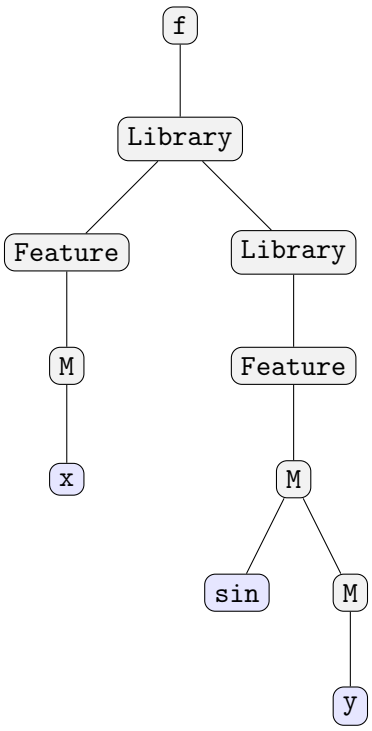

Figure 1: Abstract Syntax Tree generated by the modified grammar. The parser recursively traverses the `Library` nodes to extract the isolated `Feature` subtrees (shaded path endpoints), ultimately compiling the basis library $\mathbf{\Phi}(X) = [x, \sin(y)]$.

The structural complexity penalties ($\mathcal{C}_{\text{active}}$ and $\mathcal{C}_{\text{unused}}$) introduced in Equation 4 are calculated precisely by counting the number of nodes forming each isolated `Feature` subtree in this AST.

4 MCTS Exploration Strategies for Equation Discovery

The Core Problem: Standard MCTS algorithms were built for spatial board games (Go, Chess). We are building Abstract Syntax Trees (ASTs). Math grammars have massive branching factors ($+$, $*$, sin, cos, x, y). If the tree goes deep, the combinatorial space explodes.

1. **UCT + Uniform Random Rollouts (Current Baseline)**
   - **Mechanism:** When reaching an unexpanded node, the agent finishes the equation by picking grammar rules completely at random ($1/N$ probability) until the LIFO stack is empty.
   - **Why it fails here:** The probability of a random walk successfully building a specific, physically meaningful 8-deep equation (like a coupled oscillator) is statistically zero.
   - **Result:** The agent wastes 99% of its SINDy evaluation budget scoring mathematical garbage like $\sin(\exp(\cos(x)))$ instead of exploring broad, simple basis functions.

2. **AMAF / RAVE (The "Context-Independence" Trap)**
   - **Mechanism:** All-Moves-As-First (AMAF). If a move is good, it gets a high score *everywhere* in the tree, regardless of when it was played.
   - **Why it fails here:** Math is strictly **context-dependent**.
     - Playing $M \to \sin(M)$ at the root might yield a huge reward because the data is periodic.
     - AMAF will globally memorize that "sin is good" and start blindly injecting it deep inside other branches, like $\exp(\sin(x))$.
   - **Result:** AMAF violates the strict topological hierarchy of an Abstract Syntax Tree. It will actively destroy the mathematical structure. Do not use it.

3. **SHUSS / SHOT (The Best Immediate Upgrade)**
   - **Mechanism:** Sequential Halving Applied to Trees. Replaces the blind random rollout with a multi-armed bandit tournament at the decision node.
   - **How it works (The Tournament):**
     1. At a node with K possible grammar rules, divide the rollout budget equally among all K rules.
     2. Run them, evaluate with SINDy, and **instantly delete the bottom 50%** (e.g., throwing out $M \to M/M$ because it caused a singularity).
     3. Take the remaining budget, redistribute it equally among the surviving 50%, and repeat until only 1 optimal rule remains.
   - **Why it perfectly fits:** It aggressively prunes bad mathematical operators at the very beginning of the rollout. It forces the compute budget to focus only on structurally sound sub-equations, neutralizing the exponential branching factor.

4. **PUCT (The Deep Learning / AlphaZero SOTA)**
   - **Mechanism:** Predictor + UCT. Replaces uniform rollouts entirely with a trained Neural Network (typically a Transformer).
   - **How it works:**
     - The network reads the current partial AST.
     - Outputs a **Policy** (a rigged probability distribution favoring logical math rules, e.g., heavily biasing $M \to x$ right after a Feature node to keep the tree shallow).

  - Outputs a **Value** (predicting what the SINDy score will be without actually running SINDy).

- **Why it fits:** The network acts as automated "physics intuition." It completely bypasses the combinatorial explosion by looking at a partial equation and knowing instinctively which operators make physical sense next.

Actionable Takeaway: Drop AMAF/RAVE completely. Implement **SHUSS/SHOT** at the root and deep nodes to efficiently guide the tree generation before the SINDy layer evaluates the library.

5 Limitations of the Current MCTS-SINDy Integration

While the proposed hybrid approach aims to combine the flexibility of tree-based symbolic exploration with the efficiency of sparse regression, our initial implementation reveals several fundamental limitations. These issues are not merely due to hyperparameter choices but arise from a deeper mismatch between the assumptions of Monte Carlo Tree Search (MCTS) and the structure of the hybrid problem.

5.1 Mismatch Between MCTS Assumptions and Feature-Based Representation

In the original Symbolic Physics Learner (SPL), each complete trajectory in the search tree corresponds to a single mathematical expression. As a result, the reward function evaluates a well-defined object, and each decision in the tree directly contributes to constructing that expression.

In contrast, our hybrid approach modifies this paradigm: the MCTS no longer generates a single equation, but instead produces a *library of features* that is subsequently evaluated by a SINDy regression step. The reward is therefore not a function of an individual expression, but of a *set of interacting features*:

$$\mathcal{R} = f(\phi_1, \phi_2, \ldots, \phi_m).$$

This introduces a critical issue: the contribution of a single feature cannot be evaluated independently. A feature that is useless on its own may become essential when combined with others. For example, in the case of a linear dynamical system such as:

$$\dot{x} = -0.1x + y,$$

neither x nor y alone can explain the dynamics accurately, but their combination does. As a result, the MCTS receives low rewards for partial constructions and cannot identify which components are promising.

This phenomenon breaks one of the key assumptions of MCTS: that intermediate decisions can be evaluated through their impact on the final reward. In our setting, the reward is highly non-local and depends on complex interactions between features, making credit assignment extremely difficult.

5.2 Credit Assignment and Delayed Reward Problem

The difficulty described above is closely related to the classical credit assignment problem in reinforcement learning. In standard MCTS applications, the reward is observed only at the end of a rollout, but it can still be meaningfully attributed to earlier decisions because the structure being built is coherent and incremental.

In our hybrid setting, however, the reward is produced after a SINDy regression step that may discard most of the generated features. As a consequence:

- Features that are not selected by SINDy still influence the reward indirectly.
- The MCTS does not observe which features were actually useful.
- The same feature may lead to very different rewards depending on the rest of the library.

This results in a highly noisy and non-stationary reward signal, which significantly degrades the learning dynamics of the MCTS.

5.3 Instability Induced by Maximum Reward Backpropagation

Following SPL, our implementation uses the maximum reward observed during simulations to update node values:

$$Q(s) \leftarrow \max(Q(s), r).$$

While this strategy is effective when rewards are stable and directly tied to a single expression, it becomes problematic in our context. Due to the variability introduced by feature interactions and regression, a single high-reward rollout may be the result of a lucky combination rather than a robust pattern. Propagating this maximum value biases the search toward unstable regions of the space and reduces exploration.

In contrast, averaging rewards would provide a more reliable estimate of the expected quality of a node, especially in the presence of stochastic evaluations.

5.4 Inadequate Complexity Measure

Another limitation lies in the definition of the structural complexity penalty. In the current implementation, complexity is approximated by the length of the production rule sequence:

$$\mathcal{C}_{\text{active}} = |\text{sequence}|.$$

However, this measure does not accurately reflect the true complexity of the generated features. Two sequences of similar length may produce expressions of vastly different interpretability and usefulness. Moreover, this global measure does not distinguish between features that are retained by SINDy and those that are discarded.

As a result, the complexity penalty does not effectively enforce parsimony in the final model.

5.5 Comparison with the SPL Framework

These limitations highlight a fundamental difference between our approach and SPL. In SPL:

- Each tree corresponds to a single equation.
- The reward directly evaluates this equation.
- Continuous optimization is used to fit constants within a fixed structure.

In our hybrid model:

- Each tree generates a set of candidate features.
- The final model is obtained through a separate regression step.
- The reward evaluates a combination of features rather than a single structured expression.

While this design removes the costly continuous optimization step present in SPL, it fundamentally alters the learning problem faced by the MCTS. The search is no longer over self-contained expressions but over sets of interacting components, making the optimization landscape significantly more complex.

5.6 Summary

In summary, the current hybrid formulation introduces several challenges:

- A non-factorizable reward function depending on feature interactions.
- Severe credit assignment issues due to the indirect role of SINDy.

- Instability caused by maximum reward backpropagation.
- An inadequate measure of structural complexity.

These observations suggest that directly combining MCTS and SINDy requires a careful redesign of the reward function and the search strategy. In particular, providing more localized feedback to the MCTS agent and better aligning the representation with the evaluation mechanism appear to be crucial directions for improvement.